

FocusBloom 모바일 앱 프로그래밍 최종과제

보고서

Kotlin Android 학습 루틴 관리 앱 구현 및 코드 설명

주요 파일: app/src/main/java/com/example/malast_project/MainActivity.kt

1. 프로젝트 개요

앱 이름은 FocusBloom 이다. FocusBloom 은 사용자가 학습할 일을 등록하고, 선택한 할 일에 대해 집중 타이머를 실행하며, 완료된 세션 기록을 바탕으로 학습 통계와 추천 집중 시간을 보여주는 Kotlin Android 앱이다.

단순한 할 일 목록 앱에서 끝나지 않도록, 본 프로젝트는 타이머, 로컬 저장, 통계 계산, 추천 알고리즘, 배지 기능을 함께 구현했다. 이를 통해 사용자는 오늘 해야 할 일을 관리하면서 실제 집중 시간도 기록할 수 있다.

2. 핵심 기능 목록

- 화면 구성 기능: Kotlin 코드에서 직접 화면을 구성한다. 타이머 영역, 할 일 입력 영역, 할 일 목록 영역, 성과 분석 영역으로 나뉜다.
- 할 일 등록 기능: 사용자가 할 일 제목과 집중 시간을 입력할 수 있고, 입력한 할 일은 앱 내부 목록에 추가된다.
- 집중 타이머 기능: 선택한 할 일의 목표 시간만큼 카운트다운을 실행하고, 남은 시간과 진행률 바를 갱신한다.
- 세션 완료 처리 기능: 타이머가 끝나면 선택된 할 일을 완료 상태로 바꾸고, 완료 세션 수와 누적 집중 시간을 저장한다.
- 로컬 데이터 저장 기능: 앱을 종료해도 할 일 목록과 학습 기록이 유지되도록 SharedPreferences 와 JSON 을 사용한다.
- 성과 분석 기능: 완료 세션 수, 누적 집중 시간, 평균 세션, 연속 학습일을 보여준다.
- 추천 집중 시간 기능: 남은 할 일 수와 기존 평균 집중 시간을 기준으로 다음 집중 시간을 추천한다.

- 배지 기능: 일정 조건을 만족하면 성취 배지 문구를 보여준다.

3. 주요 코드 파일

핵심 코드는 MainActivity.kt 파일에 들어 있다. 이 파일 하나에 화면 구성, 타이머 동작, 할 일 관리, 데이터 저장, 통계 계산이 모두 구현되어 있다.

`app/src/main/java/com/example/malast_project/MainActivity.kt`

4. 기능별 코드 설명

아래 표는 앱의 핵심 기능이 어떤 함수에 구현되어 있는지 정리한 것이다. 이후 각 항목에 대해 구현 의도를 설명한다.

기능	관련 코드	역할
앱 시작 및 초기화	onCreate()	앱 실행 시 저장소를 준비하고, 기존 데이터를 불러온 뒤 화면을 구성한다.
화면 구성	buildUi(), card(), row(), text(), input()	ScrollView와 LinearLayout을 이용해 전체 화면과 반복 UI 컴포넌트를 만든다.
할 일 등록	addTask(), FocusTask	입력값을 검사하고 FocusTask 객체를 생성해 목록에 추가한다.
집중 시간 입력 제한	parseMinutes()	입력 시간을 숫자로 변환하고 5분에서 90분 사이로 제한한다.
타이머 실행	startTimer()	CountDownTimer를 생성해 1초마다 남은 시간을 갱신한다.
타이머 화면 갱신	updateTimerView()	남은 시간을 mm:ss로 표시하고 ProgressBar 값을 계산한다.
타이머 초기화	resetTimer()	선택된 할 일의 목표 시간으로 타이머를 다시 설정한다.
세션 완료 처리	completeSession()	할 일을 완료 상태로 바꾸고 통계 값을 저장한다.
화면/통계 갱신	refresh()	목록, 추천 시간, 통계, 배지 문구를 최신 데이터로 다시 표시한다.

추천 시간 계산	recommendMinutes()	남은 할 일 수와 평균 집중 시간을 기준으로 추천 시간을 반환한다.
연속 학습일 계산	updateStreak(), isYesterday()	오늘 날짜와 마지막 학습 날짜를 비교해 연속 학습일을 계산한다.
데이터 저장/불러오기	saveTasks(), loadTasks()	할 일 목록을 JSON 문자열로 저장하고 다시 객체 목록으로 복원한다.
기본 데이터 생성	seedTasks()	첫 실행 시 예시 할 일을 만들어 빈 화면을 방지한다.

4.1 앱 시작 및 초기화

onCreate()는 앱이 처음 실행될 때 호출된다. SharedPreferences 를 준비하고, 저장된 할 일 목록을 불러온 뒤, 데이터가 없으면 seedTasks()로 기본 예시 할 일을 생성한다. 이후 buildUi()로 화면을 만들고 refresh()로 화면에 데이터를 표시한다.

4.2 화면 구성 기능

buildUi()는 앱 화면 전체를 만드는 함수이다. XML 레이아웃 대신 Kotlin 코드로 ScrollView, LinearLayout, TextView, Button, EditText, ProgressBar 를 직접 생성한다. card(), row(), text() 같은 보조 함수는 반복되는 UI 코드를 줄이기 위해 사용했다.

4.3 할 일 등록 기능

addTask()는 사용자가 입력한 제목과 시간을 검사한 뒤 FocusTask 객체를 만들어 tasks 리스트에 추가한다. 제목이 비어 있으면 Toast 메시지로 안내하고, 정상 입력이면 saveTasks()와 refresh()를 호출해 저장과 화면 갱신을 처리한다.

4.4 집중 타이머 기능

startTimer()는 CountdownTimer 를 사용해 카운트다운을 시작한다. onTick()에서는 remainingMs 를 갱신하고 updateTimerView()를 호출한다. updateTimerView()는 남은 시간을 mm:ss 형식으로 표시하고 ProgressBar 진행률을 계산한다.

4.5 세션 완료 처리 기능

completeSession()은 타이머가 끝났을 때 실행된다. 선택된 할 일을 완료 상태로 바꾸고, 누적 집중 시간과 완료 세션 수를 SharedPreferences 에 저장한다. 또한 updateStreak()로 연속 학습일을 갱신한다.

4.6 화면 갱신 및 통계 표시

refresh()는 현재 데이터를 화면에 다시 표시한다. 할 일 목록을 다시 그리고, 남은 할 일 수, 추천 집중 시간, 완료 세션 수, 누적 집중 시간, 평균 세션, 배지 문구를 한 번에 갱신한다.

4.7 추천 집중 시간 기능

recommendMinutes()는 남은 할 일 수와 평균 집중 시간을 기준으로 추천 시간을 반환한다. 남은 할 일이 많으면 짧은 15 분을 추천하고, 평균 집중 시간이 길면 30 분을 추천한다. 이 규칙 기반 로직이 단순 타이머 앱과 차별화되는 부분이다.

4.8 로컬 데이터 저장 기능

saveTasks()는 할 일 목록을 JSONArray 와 JSONObject 로 변환해 저장하고, loadTasks()는 저장된 JSON 문자열을 다시 FocusTask 객체 목록으로 복원한다. 이 구조 덕분에 앱을 종료해도 할 일 목록과 완료 여부가 유지된다.

5. 사용한 Android/Kotlin 요소

- Activity
- LinearLayout
- ScrollView
- TextView
- Button
- EditText
- ProgressBar
- CountdownTimer
- SharedPreferences
- JSONArray

- JSONObject
- Toast
- Kotlin data class
- Kotlin when

6. 어려웠던 점과 해결 방법

타이머 상태와 선택된 할 일 관리: selectedIndex, activeMinutes, remainingMs, running 변수를 분리하여 현재 선택된 할 일과 타이머 상태를 안정적으로 관리했다.

여러 할 일의 영구 저장: 단순 문자열 저장으로는 목록 관리가 어렵기 때문에 JSONArray 와 JSONObject 를 사용해 구조화된 JSON 문자열로 저장했다.

추천 기능 구현 범위 조절: 복잡한 AI 모델 대신 규칙 기반 알고리즘을 사용하여 과제 규모에 맞고 설명하기 쉬운 방식으로 개인화 기능을 구현했다.

7. 독창적인 부분

FocusBloom 은 단순히 할 일을 체크하는 앱이 아니라, 사용자의 학습 기록을 바탕으로 다음 집중 시간을 추천한다. 또한 완료 세션 수, 누적 집중 시간, 연속 학습일, 배지 기능을 함께 제공하여 사용자가 자신의 학습 습관을 확인할 수 있도록 했다.

8. AI 활용 명시

본 프로젝트는 앱 주제 선정, 코드 작성, 보고서 구조화 과정에서 AI 도구의 도움을 받았다. 최종 제출 전에는 Kotlin 코드와 빌드를 직접 확인했으며, Android Studio 프로젝트에서 실행 가능한 형태로 정리했다.